

Explicit Implicit Template Regions

Document #: P3525R0 [[Latest](#)] [[Status](#)]
Date: 2024-12-16
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[2 Proposal](#)

[2.1 Wording](#)

[2.2 Feature-Test Macro](#)

[3 References](#)

§ 1 Introduction

[[P1061R9](#)] introduced the ability to declare packs inside of structured bindings and, furthermore, even proposed the ability to do so outside of any templated context:

```
struct Point { int x, y; };  
  
// not a template  
auto sum(Point p) -> int {  
    // yet here is a pack  
    auto [... parts] = p;  
  
    // that I can fold over  
    return (... + parts);  
}
```

This is useful (and implemented! and worded!), but it has some surprising consequences. In order for the feature to work in the ways that users would expect, everything after the declaration of `parts` above must become, implicitly, a template. See that paper for a more detailed description with examples. Templates have different rules than non-templates in a variety of ways, but there is not much of a marker to differentiate this.

As such, the part of that proposal that allowed packs outside of templates was ripped out and [[P1061R10](#)] was adopted in the Wrocław meeting, requiring packs in structured bindings to be declared inside of a template context. This is simpler in a way, but requires users to just... arbitrarily make their code into a template.

For instance, you want to write the above code, but you cannot. So you have to do something like this:

A Template (for no reason)	A Generic Lambda (for no reason)
<pre>template <class> auto sum(Point p) -> int { auto [... parts] = p; return (... + parts); }</pre>	<pre>auto sum(Point p) -> int { return [&<class T=void>(){ auto [... parts] = p; return (... + parts); }(); }</pre>

Both of these options are bad. Turning the whole function into a template opens up to the potential of multiple instantiations, if you want to put the definition in a header you have to explicitly instantiate the template in the source file, and so forth. Wrapping the contents in an immediately invoked, generic lambda is... better. It avoids many of the problems of the unnecessary template. But it introduces a new function scope, which interacts badly if the body of the function wants to *conditionally* return.

For example:

```
auto some_function(Point p) -> bool {
    if (/* some condition */) {
        // what I want to write is this
        auto [... parts] = p;
        if (foo(parts...)) {
            return false;
        }

        // but I would have to write something like... this?
        auto ret = [&<class T=void>() -> optional<bool> {
            auto [...parts] = p;
            if (foo(parts...)) {
                return false;
            }
            return nullopt;
        }();
        if (ret) return *ret;
    } else {
        // do some other thing
    }
}
```

Alternatively, you could preemptively make your entire body `return [&<class=void>(){ ... }();` even if only a small part of it wants to introduce a pack.

In general, I want to be able to write code that directly expresses user intent. Not come up with workarounds for not being able to do so.

§ 2 Proposal

The [P1061R9] design introduced the concept of an *implicit* template region. Let's just add an *explicit* implicit template region. We can copy the syntactic idea of a `constexpr` block as introduced in [P3289R0]:

```
auto sum(Point p) -> int {
    auto [... bad_parts] = p; // error: not in a template

    template {
        auto [... good_parts] = p; // OK, in a template (explicitly)
        return (... + good_parts);
    }
}
```

We still have to be explicit about being in a template, but we can do so in a significantly more lightweight way: the template region is localized to the function body (as in [P1061R9]) and without introducing an extra function scope that interferes with returns and coroutines (also as in [P1061R9]).

I believe this addresses all the implementor concerns with the original design. It also provides a path forward to eventually removing the block, if so desired. It additionally provides a path forward to answering more complicated questions with other language features (like member packs [P3115R0]).

§ 2.1 Wording

Extend the grammar for statement in 8.1 [stmt.pre]:

```
statement:
    labeled-statement
    attribute-specifier-seqopt expression-statement
    attribute-specifier-seqopt compound-statement
+   attribute-specifier-seqopt template-block
    attribute-specifier-seqopt selection-statement
    attribute-specifier-seqopt iteration-statement
    attribute-specifier-seqopt jump-statement
    declaration-statement
    attribute-specifier-seqopt try-block
```

And a corresponding new clause after 8.4 [stmt.block], call it [stmt.template]:

```
template-block:
    template compound-statement
```

A *template block* introduces an explicit template region ([temp.pre]) encompassing the block scope introduced by the *compound-statement*.

[Example 1:

```

struct Point { int x, y; };

int magnitude(Point p) {
    template {
        auto [...good] = p; // OK, within explicit template region
        return (good * good + ...);
    }

    auto [...bad] = p; // error: bad is not a templated entity
}

— end example ]

```

Mark all entities within an an explicit template region as templated, in 13.1 [\[temp.pre\]](#):

- 8 An entity is *templated* if it is
- (8.1) — a template,
 - (8.2) — an entity defined ([basic.def]) or created ([class.temporary]) in a templated entity,
 - (8.3) — a member of a templated entity,
 - (8.4) — an enumerator for an enumeration that is a templated entity, ~~or~~
 - (8.5) — the closure type of a lambda-expression ([expr.prim.lambda.closure]) appearing in the declaration of a templated entity- , or
 - (8.6) — an entity defined or created within an explicit template region ([stmt.template]).
- [Note 1: A local class, a local or block variable, or a friend function defined in a templated entity is a templated entity. — end note]

Which needs a point of instantiation at the end of 13.8.4.1 [\[temp.point\]](#):

- * For an explicit template region, the point of instantiation immediately follows the closing brace of the *compound-statement* of the *template-block*.

§ 2.2 Feature-Test Macro

Introduce a new `__cpp_template_block` to 15.11 [\[cpp.predefined\]](#):

```
+ __cpp_template_block 2025XXL
```

§ 3 References

[P1061R10] Barry Revzin and Jonathan Wakely. 2024-11-22. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r10>

[P1061R9] Barry Revzin, Jonathan Wakely. 2024-08-24. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r9>

[P3115R0] Corentin Jabot. 2024-02-15. Data Member, Variable and Alias Declarations Can Introduce A Pack.

<https://wg21.link/p3115r0>

[P3289R0] Daveed Vandevoorde, Wyatt Childers, Barry Revzin. 2024-05-21. Consteval blocks.

<https://wg21.link/p3289r0>